```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier

# XOR dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0])

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=2000,
    random_state=42
)

model.fit(X, y)

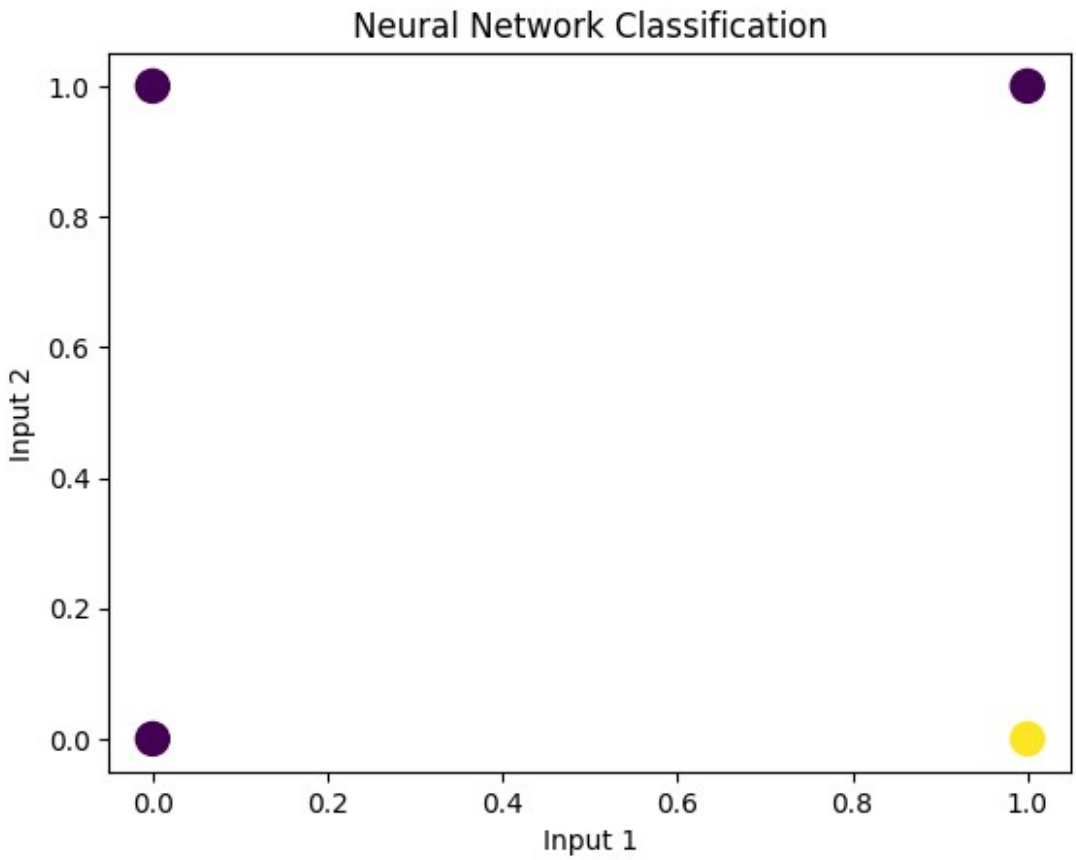
predictions = model.predict(X)

print("Actual:", y)
print("Predicted:", predictions)

plt.scatter(X[:, 0], X[:, 1], c=predictions, s=150)

plt.xlabel("Input 1")
plt.ylabel("Input 2")
plt.title("Neural Network Classification")
plt.show()
```

```
Actual: [0 1 1 0]
Predicted: [0 0 1 0]
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data[:, :2]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

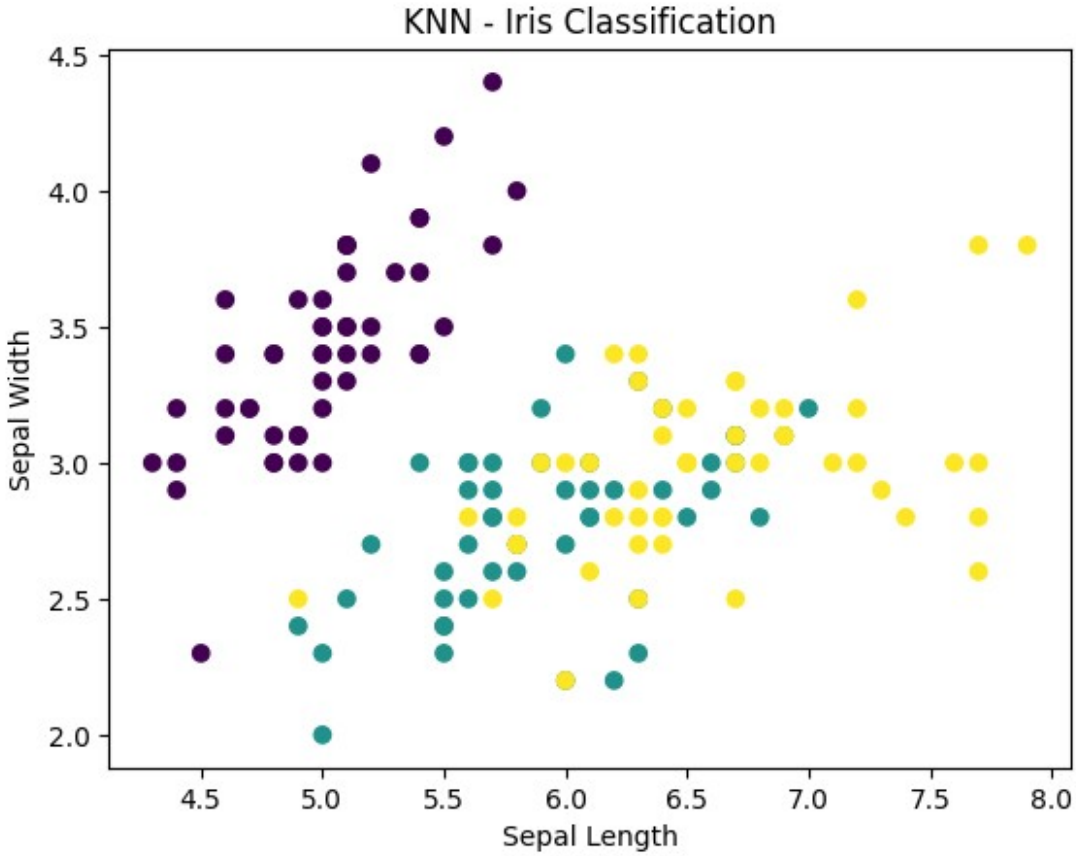
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(X[:, 0], X[:, 1], c=y)
```

```
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("KNN - Iris Classification")
plt.show()
```

```
Accuracy: 0.8
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

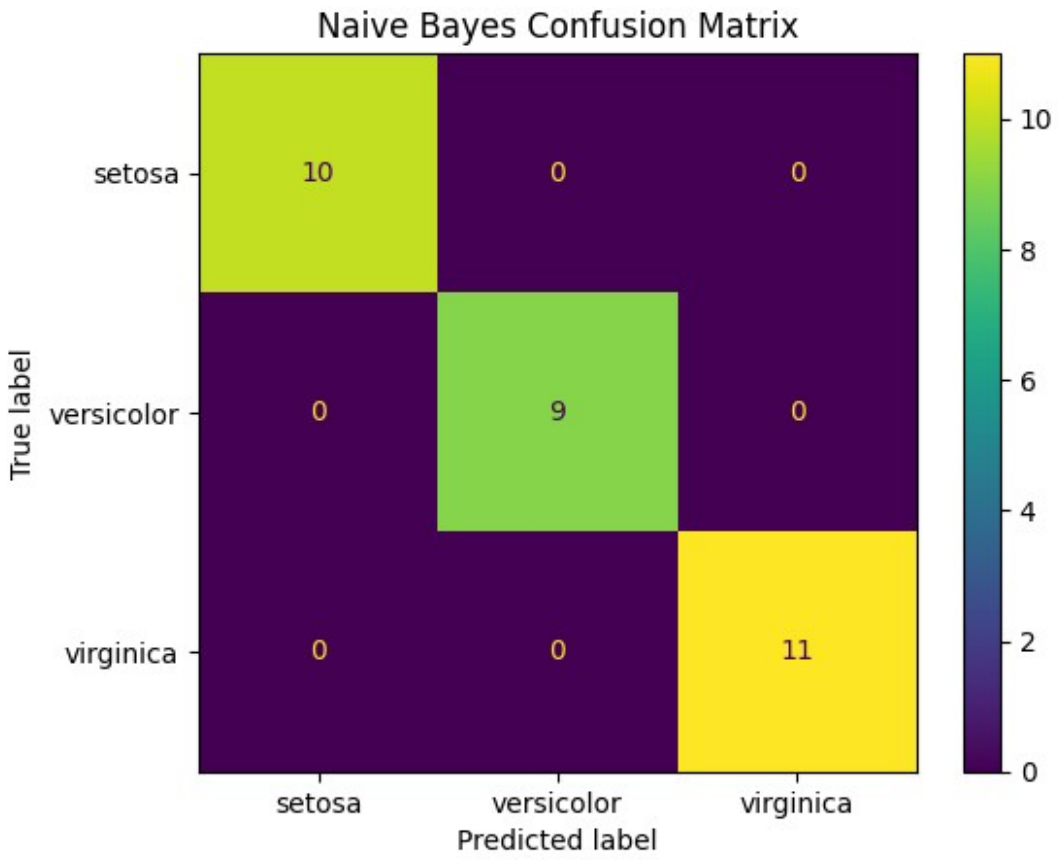
print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)

disp.plot()
plt.title("Naive Bayes Confusion Matrix")
plt.show()
```

```
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data[:, :2]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

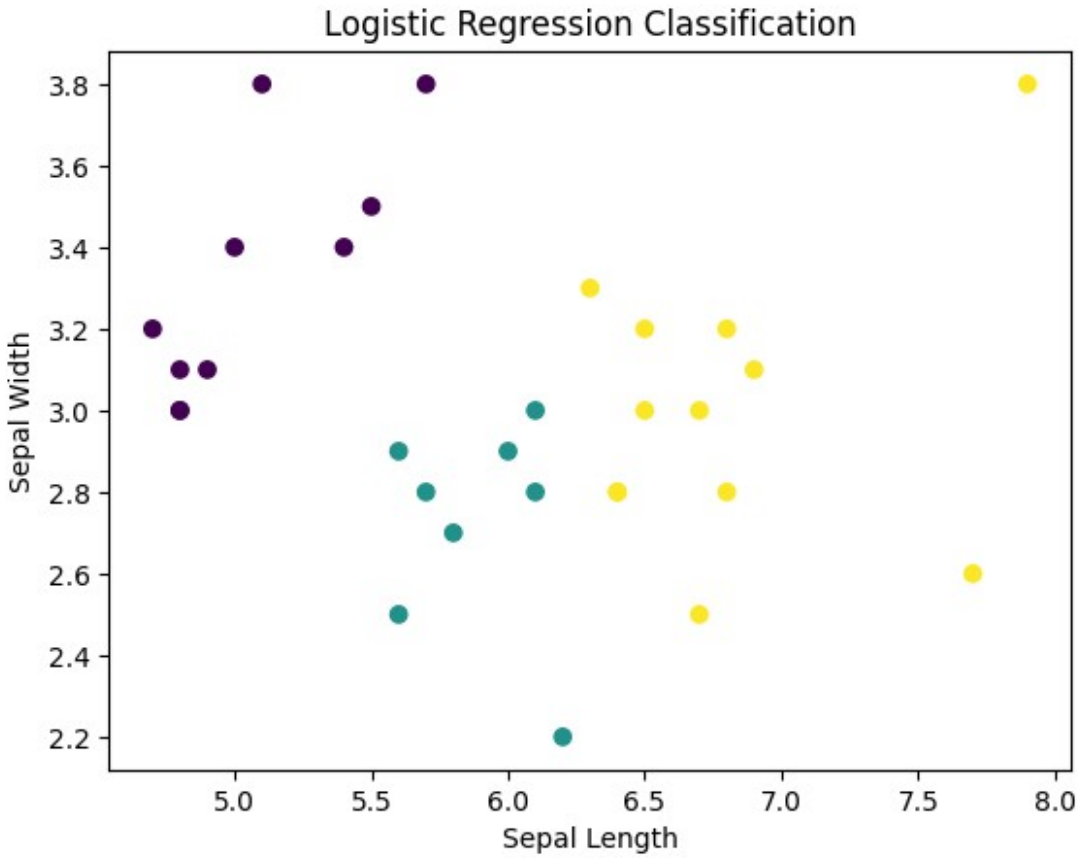
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred)
```

```
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Logistic Regression Classification")
plt.show()

Accuracy: 0.9
```




```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5],
              [6], [7], [8], [9], [10]])

y = np.array([2, 4, 5, 8, 10,
              12, 14, 16, 18, 20])

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)
```

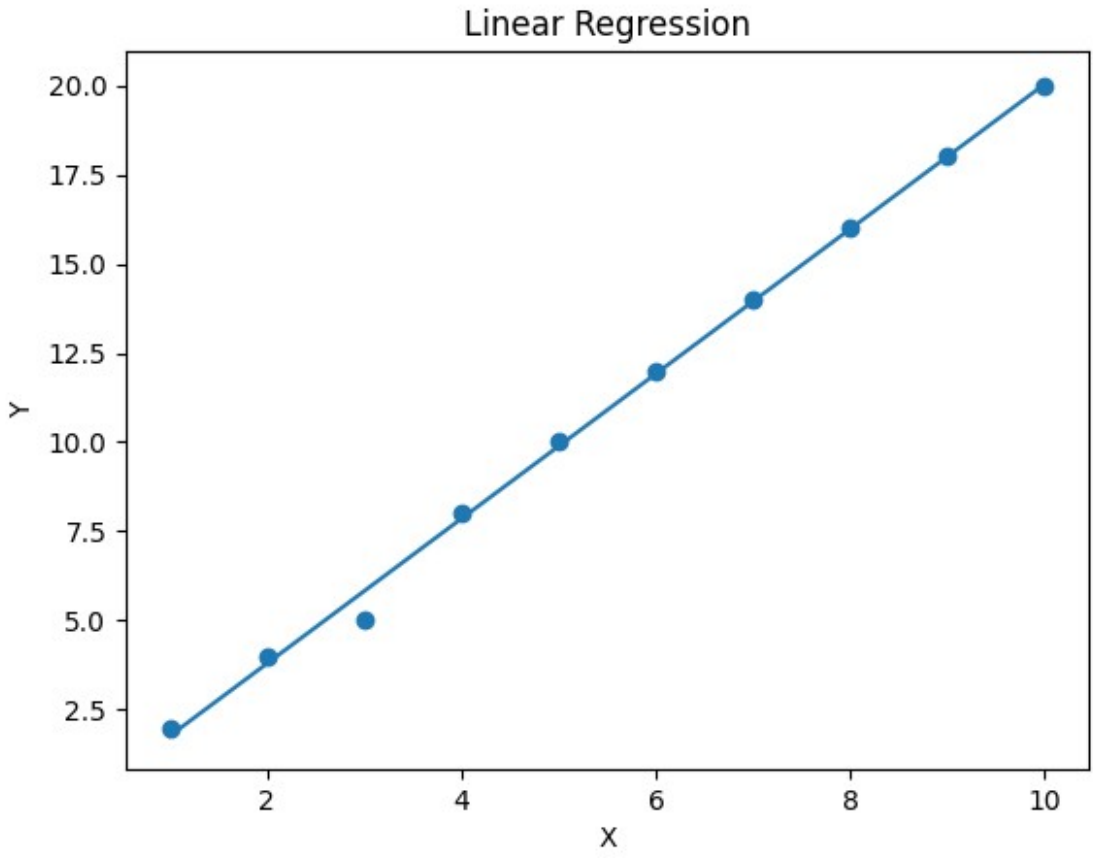
```
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)

print("Prediction for X=11:",
      model.predict([[11]])[0])

plt.scatter(X, y)
plt.plot(X, y_pred)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

```
Slope: 2.0303030303030307
Intercept: -0.26666666666666927
Prediction for X=11: 22.06666666666667
```




```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
X = np.array([[1], [2], [3], [4], [5],
              [6], [7], [8], [9], [10]])

y = np.array([3, 5, 8, 12, 18,
              25, 35, 48, 65, 85])

# Linear Regression
linear_model = LinearRegression()
linear_model.fit(X, y)

linear_pred = linear_model.predict(X)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)

poly_pred = poly_model.predict(X_poly)

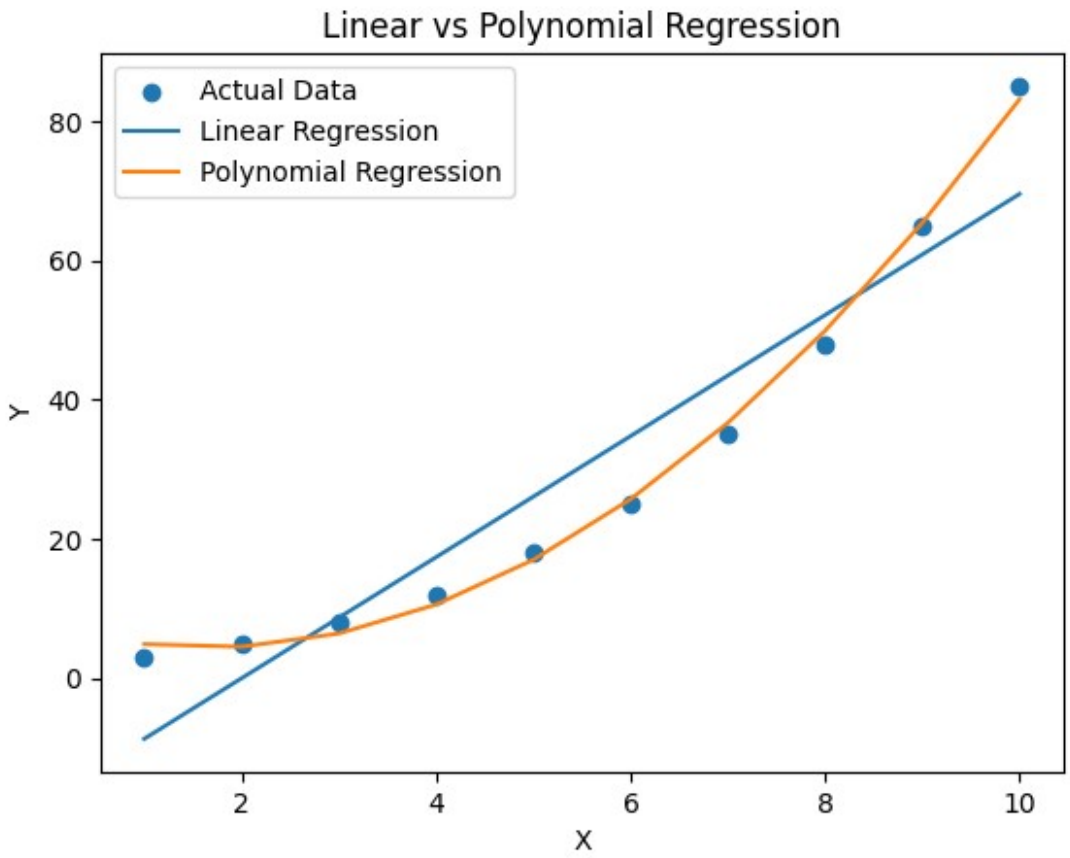
print("Linear R²:",
      linear_model.score(X, y))

print("Polynomial R²:",
      poly_model.score(X_poly, y))

plt.scatter(X, y, label="Actual Data")
plt.plot(X, linear_pred, label="Linear Regression")
plt.plot(X, poly_pred, label="Polynomial Regression")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear vs Polynomial Regression")
plt.legend()
plt.show()
```

```
Linear R²: 0.8993969485280611
Polynomial R²: 0.997151163502702
```




```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

np.random.seed(42)

X1 = np.random.normal(2, 0.5, 100)
X2 = np.random.normal(7, 0.8, 100)

X = np.concatenate([X1, X2]).reshape(-1, 1)

model = GaussianMixture(
    n_components=2,
    random_state=42
)

model.fit(X)

labels = model.predict(X)

print("Cluster Means:")
print(model.means_)
```

```
print("\nCluster Covariances:")
print(model.covariances_)

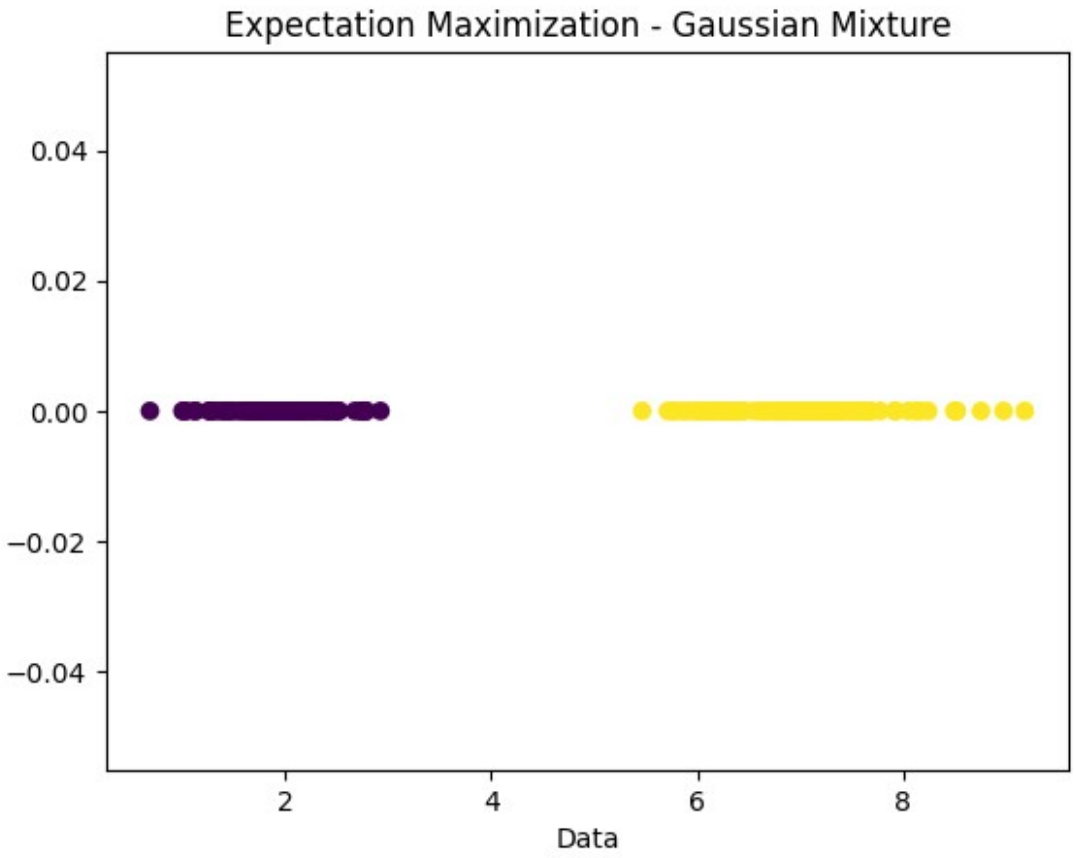
plt.scatter(X[:, 0], np.zeros(len(X)), c=labels)

plt.xlabel("Data")
plt.title("Expectation Maximization - Gaussian Mixture")
plt.show()
```

```
Cluster Means:
[[1.94807669]
 [7.01784341]]

Cluster Covariances:
[[[0.20413151]]

 [[0.57625144]]]
```




```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

data = {
    'Age': [25, 35, 45, 23, 52, 40, 30, 60, 28, 50],
    'Income': [25000, 50000, 70000, 20000, 90000,
               65000, 40000, 100000, 30000, 80000],
    'Debt': [10000, 15000, 12000, 18000, 10000,
             8000, 20000, 5000, 16000, 7000],
    'CreditScore': ['Poor', 'Good', 'Good', 'Poor', 'Excellent',
                    'Good', 'Poor', 'Excellent', 'Poor', 'Excellent']
}

df = pd.DataFrame(data)

le = LabelEncoder()
df['CreditScore'] = le.fit_transform(df['CreditScore'])

X = df[['Age', 'Income', 'Debt']]
y = df['CreditScore']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

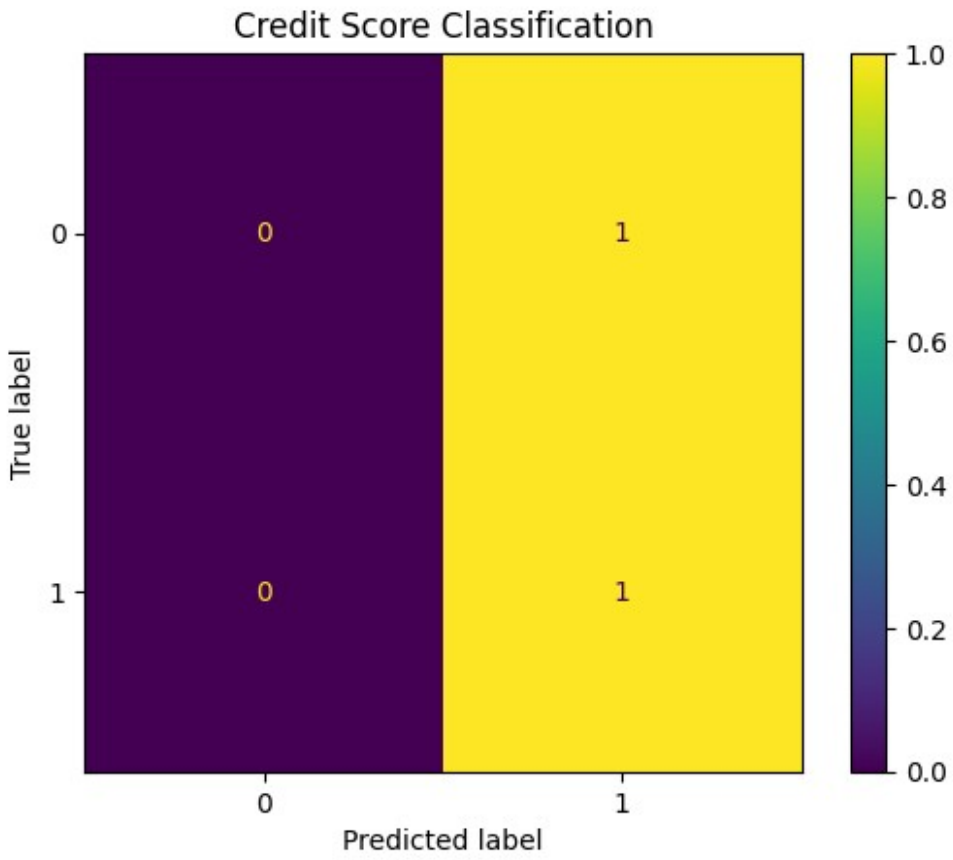
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

ConfusionMatrixDisplay(cm).plot()
plt.title("Credit Score Classification")
plt.show()
```

```
Accuracy: 0.5
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

prediction = model.predict(X_test)

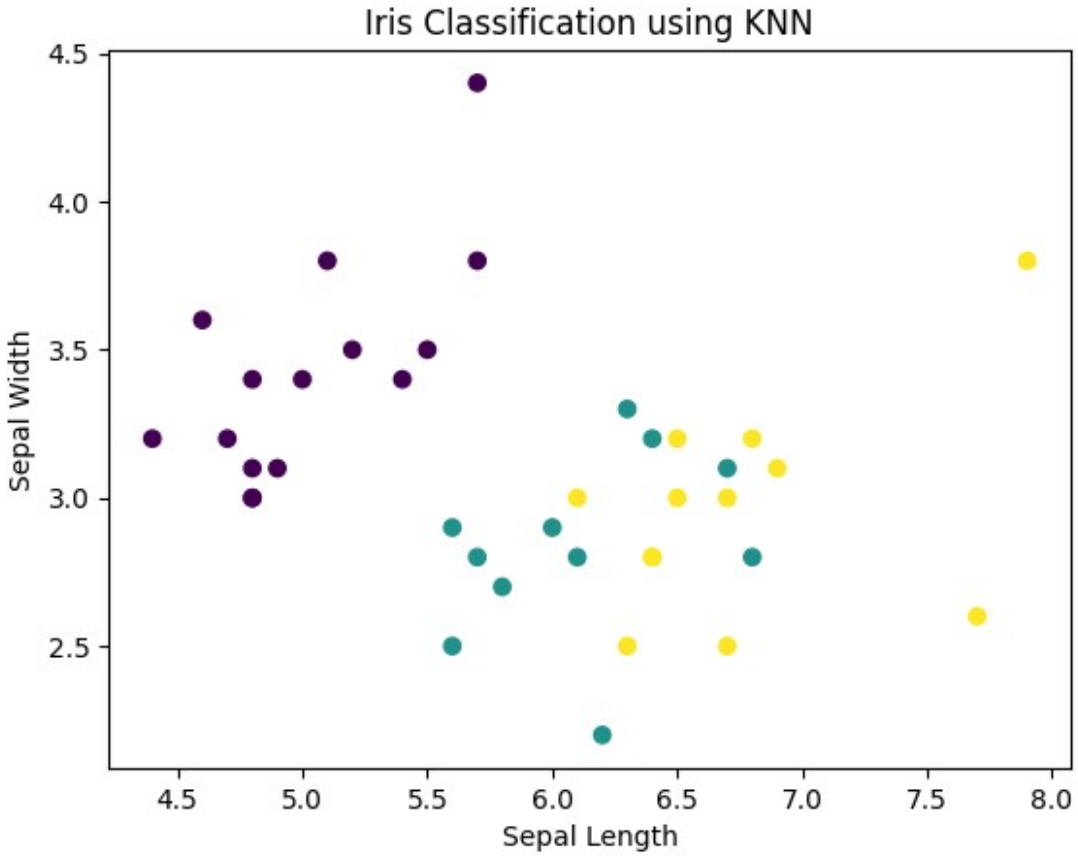
print("Accuracy:", accuracy_score(y_test, prediction))

plt.scatter(
```

```
    X_test[:, 0],
    X_test[:, 1],
    c=prediction
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Iris Classification using KNN")
plt.show()

Accuracy: 1.0
```




```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = {
    'Age': [1, 2, 3, 4, 5, 6, 7, 8],
    'Kilometers': [10000, 20000, 30000, 40000,
                   50000, 60000, 70000, 80000],
    'Price': [900000, 820000, 750000, 680000,
              600000, 530000, 450000, 380000]
}
```

```
df = pd.DataFrame(data)

X = df[['Age', 'Kilometers']]
y = df['Price']

model = LinearRegression()
model.fit(X, y)

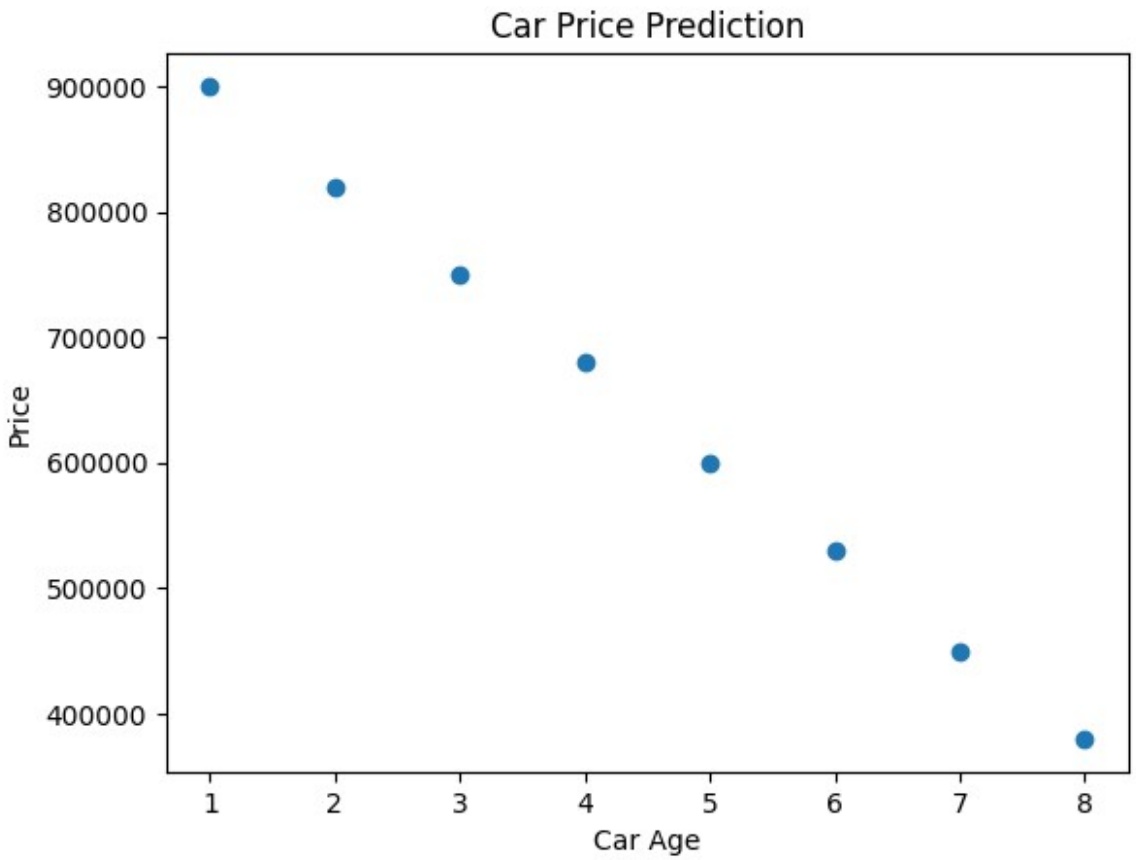
prediction = model.predict([[4, 40000]])

print("Predicted Car Price:", prediction[0])

plt.scatter(df['Age'], df['Price'])
plt.xlabel("Car Age")
plt.ylabel("Price")
plt.title("Car Price Prediction")
plt.show()
```

```
Predicted Car Price: 675833.3333333335
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(
```




```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = {
    'Area': [800, 1000, 1200, 1500, 1800,
             2000, 2200, 2500],
    'Bedrooms': [2, 2, 3, 3, 3, 4, 4, 4],
    'Price': [2500000, 3000000, 3500000, 4200000,
              5000000, 5800000, 6500000, 7500000]
}

df = pd.DataFrame(data)

X = df[['Area', 'Bedrooms']]
y = df['Price']

model = LinearRegression()
model.fit(X, y)

area = 1600
bedrooms = 3
```

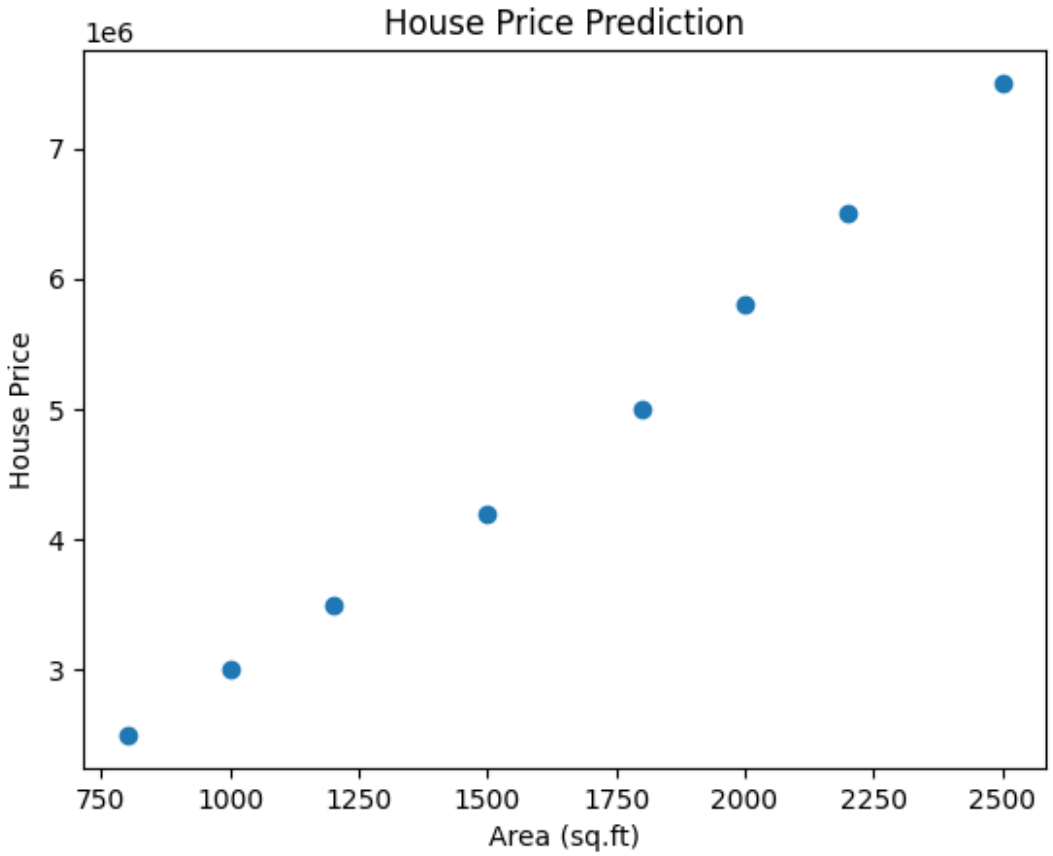
```
prediction = model.predict([[area, bedrooms]])

print("Predicted House Price:", prediction[0])

plt.scatter(df['Area'], df['Price'])
plt.xlabel("Area (sq.ft)")
plt.ylabel("House Price")
plt.title("House Price Prediction")
plt.show()
```

```
Predicted House Price: 4678483.920367534

/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

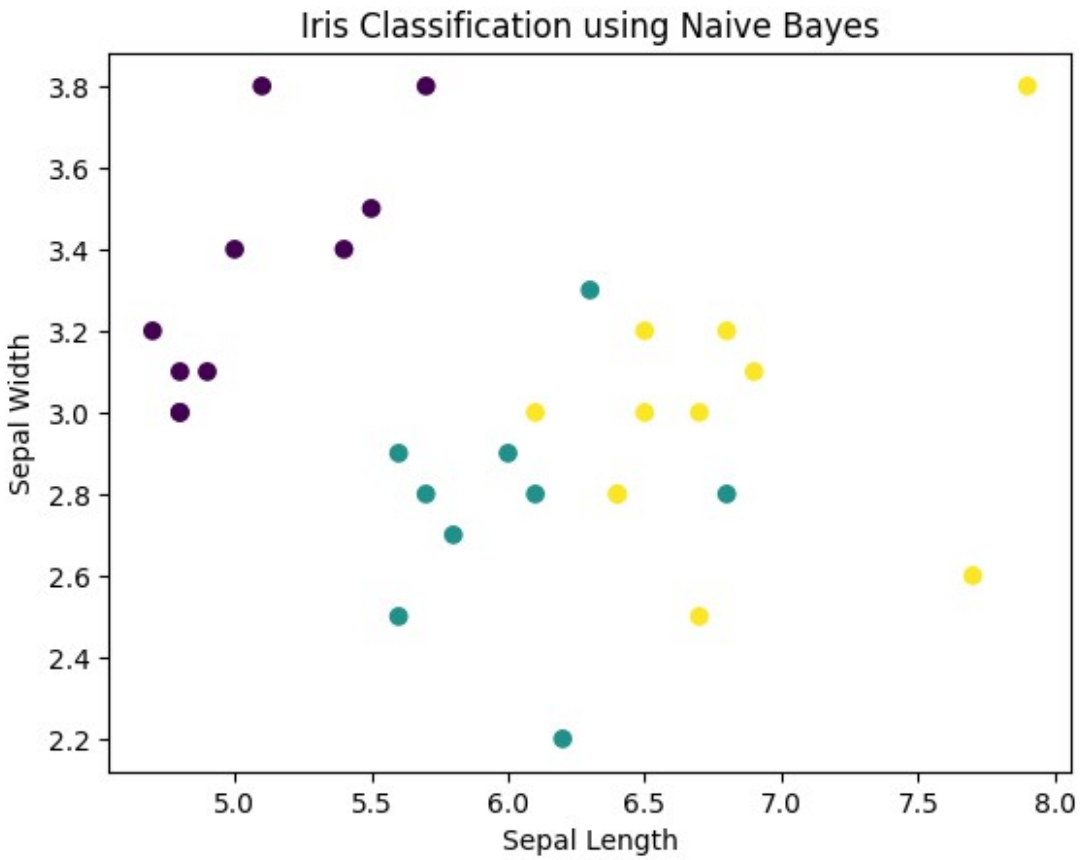
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=y_pred
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Iris Classification using Naive Bayes")
plt.show()
```

```
Accuracy: 1.0
```




```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42)
}

results = {}

for name, model in models.items():

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    accuracy = accuracy_score(y_test, prediction)

    results[name] = accuracy

    print(name, ":", accuracy)

plt.figure(figsize=(10, 5))

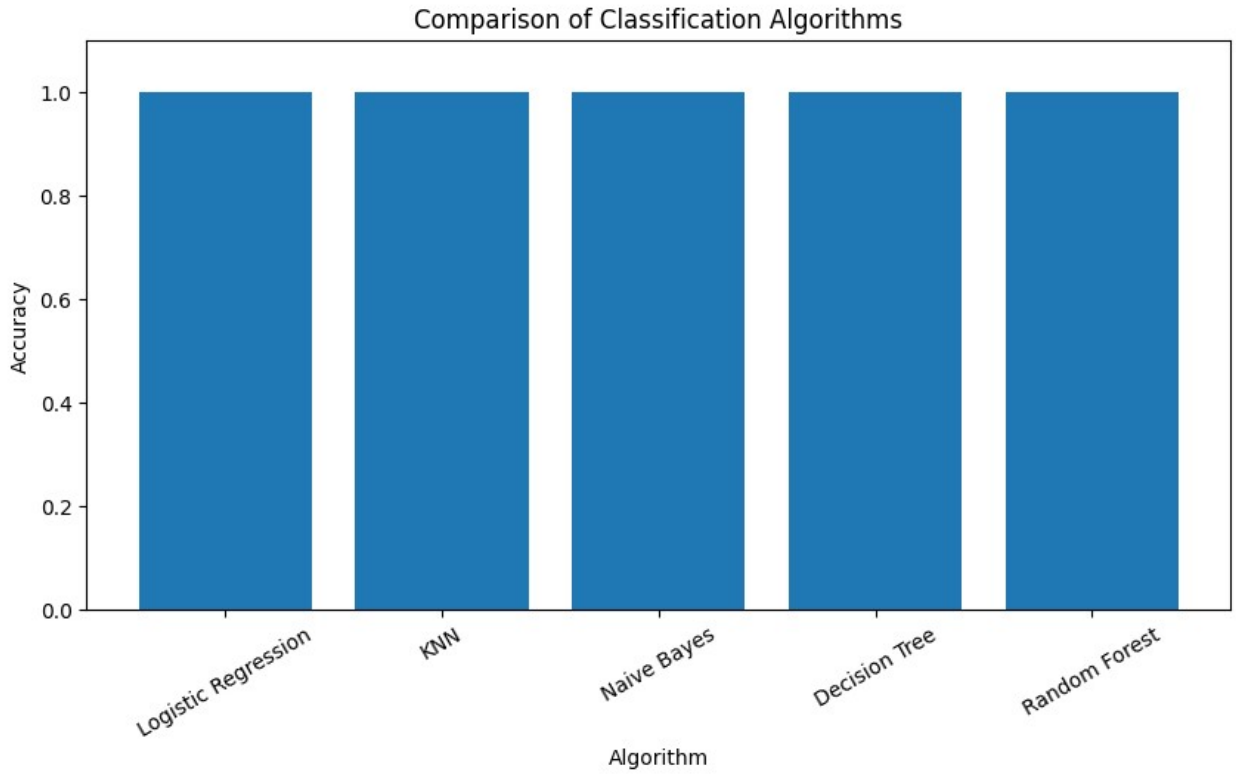
plt.bar(results.keys(), results.values())

plt.ylabel("Accuracy")
plt.xlabel("Algorithm")
plt.title("Comparison of Classification Algorithms")

plt.xticks(rotation=30)
plt.ylim(0, 1.1)

plt.show()
```

```
Logistic Regression : 1.0
KNN : 1.0
Naive Bayes : 1.0
Decision Tree : 1.0
Random Forest : 1.0
```




```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    'RAM': [2, 2, 3, 3, 4, 4, 6, 6, 8, 8],
    'Battery': [2500, 3000, 3000, 3500, 3500,
                4000, 4000, 4500, 4500, 5000],
    'Storage': [32, 32, 64, 64, 64,
                128, 128, 128, 256, 256],
    'PriceRange': [0, 0, 1, 1, 1,
                   2, 2, 2, 3, 3]
}

df = pd.DataFrame(data)

X = df[['RAM', 'Battery', 'Storage']]
y = df['PriceRange']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

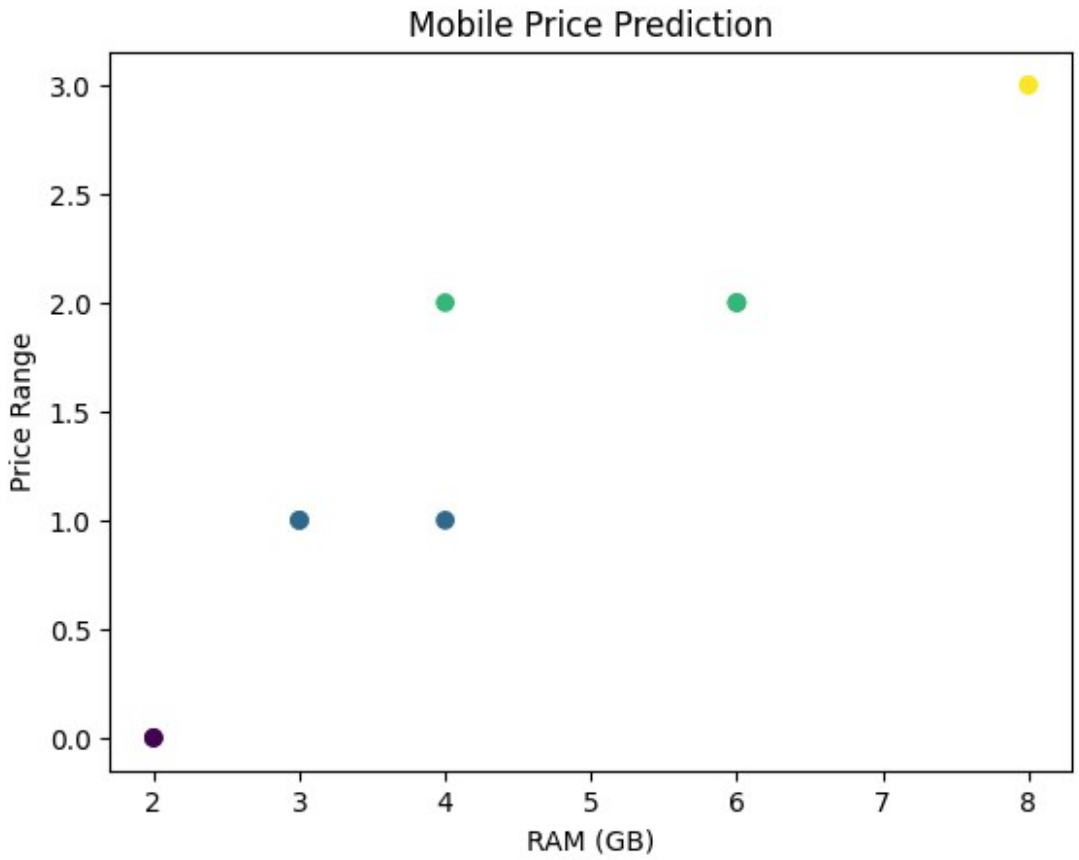
print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(df['RAM'], df['PriceRange'], c=df['PriceRange'])

plt.xlabel("RAM (GB)")
plt.ylabel("Price Range")
plt.title("Mobile Price Prediction")

plt.show()
```

```
Accuracy: 0.5
```




```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

iris = load_iris()
```

```
# Use only two classes
X = iris.data[:100, :2]
y = iris.target[:100]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

model = Perceptron(max_iter=1000, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

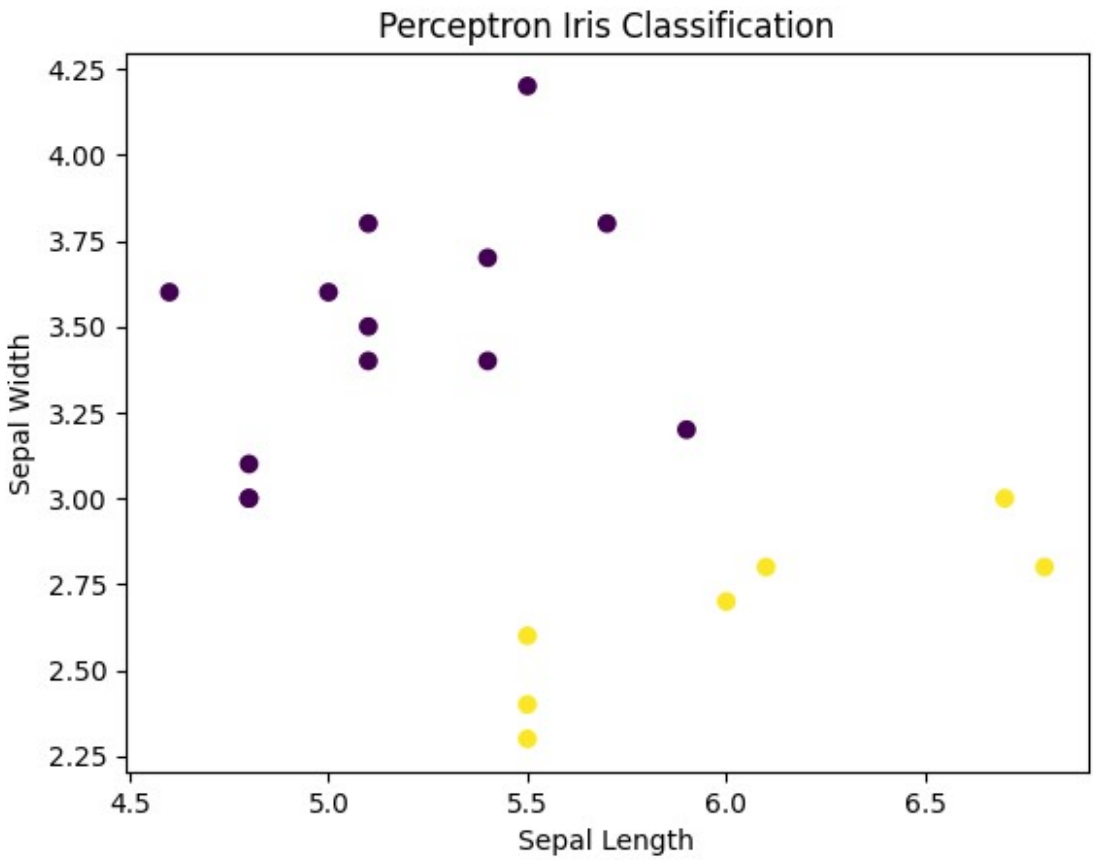
print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=y_pred
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Perceptron Iris Classification")

plt.show()
```

```
Accuracy: 0.95
```




```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    'Age': [25, 35, 45, 28, 50, 40, 30, 55, 32, 48],
    'Income': [25000, 50000, 80000, 30000, 90000,
               70000, 40000, 100000, 45000, 85000],
    'LoanAmount': [100000, 200000, 300000, 120000,
                   400000, 250000, 150000, 500000,
                   180000, 350000],
    'LoanApproved': ['No', 'Yes', 'Yes', 'No', 'Yes',
                     'Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

le = LabelEncoder()
```

```
df['LoanApproved'] = le.fit_transform(
    df['LoanApproved']
)

X = df[['Age', 'Income', 'LoanAmount']]
y = df['LoanApproved']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

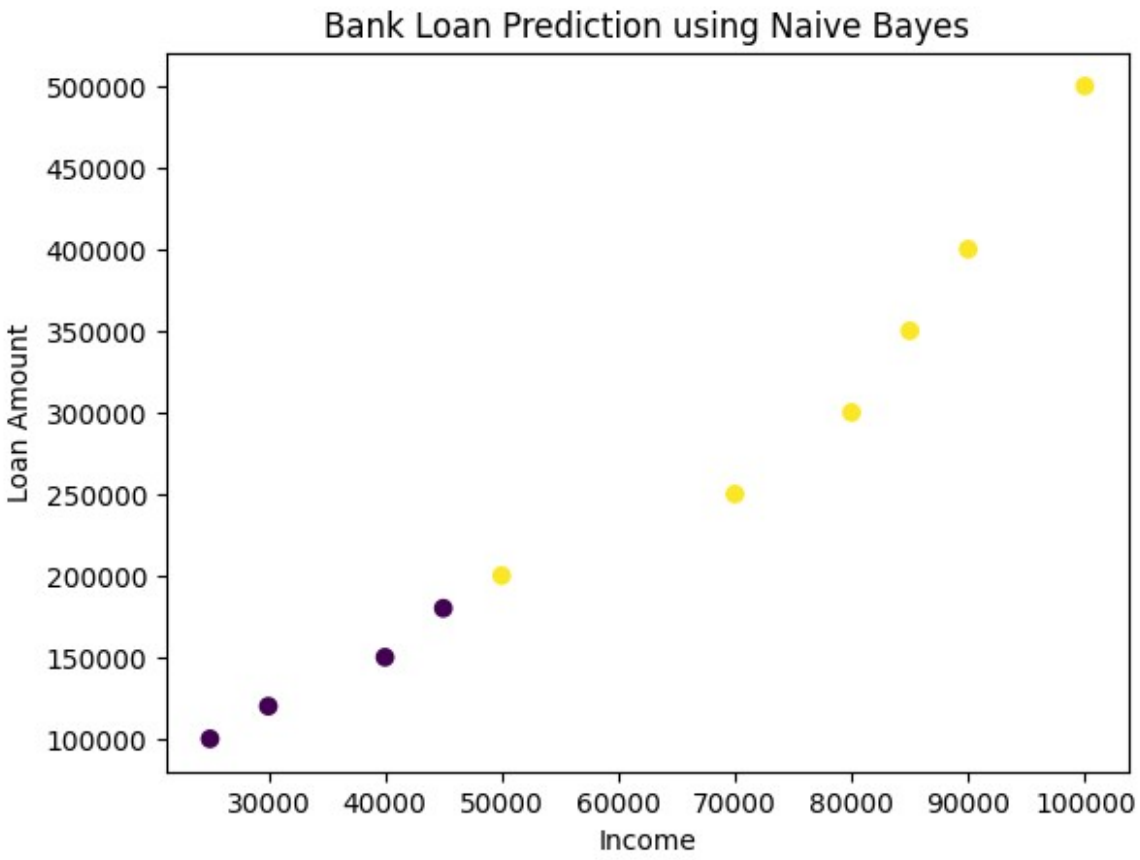
print("Accuracy:",
      accuracy_score(y_test, y_pred))

plt.scatter(
    df['Income'],
    df['LoanAmount'],
    c=df['LoanApproved']
)

plt.xlabel("Income")
plt.ylabel("Loan Amount")
plt.title("Bank Loan Prediction using Naive Bayes")

plt.show()
```

```
Accuracy: 1.0
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

# Historical sales data
data = {
    'Month': np.arange(1, 13),
    'Sales': [
        12000, 13500, 15000, 16000,
        17500, 19000, 20500, 22000,
        23500, 25000, 27000, 29000
    ]
}

df = pd.DataFrame(data)

# Input and output
X = df[['Month']]
y = df['Sales']

# Create model
```

```
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict historical sales
df['Predicted Sales'] = model.predict(X)

# Predict next 6 months
future_months = np.arange(13, 19).reshape(-1, 1)

future_sales = model.predict(future_months)

print("Future Sales Prediction:")
for month, sales in zip(
    range(13, 19),
    future_sales
):
    print(
        f"Month {month}: "
        f"{sales:.2f}"
    )

# Plot historical and predicted sales
plt.figure(figsize=(10, 6))

plt.scatter(
    df['Month'],
    df['Sales'],
    label='Actual Sales'
)

plt.plot(
    df['Month'],
    df['Predicted Sales'],
    label='Regression Line'
)

plt.plot(
    future_months,
    future_sales,
    marker='o',
    linestyle='--',
    label='Future Sales'
)

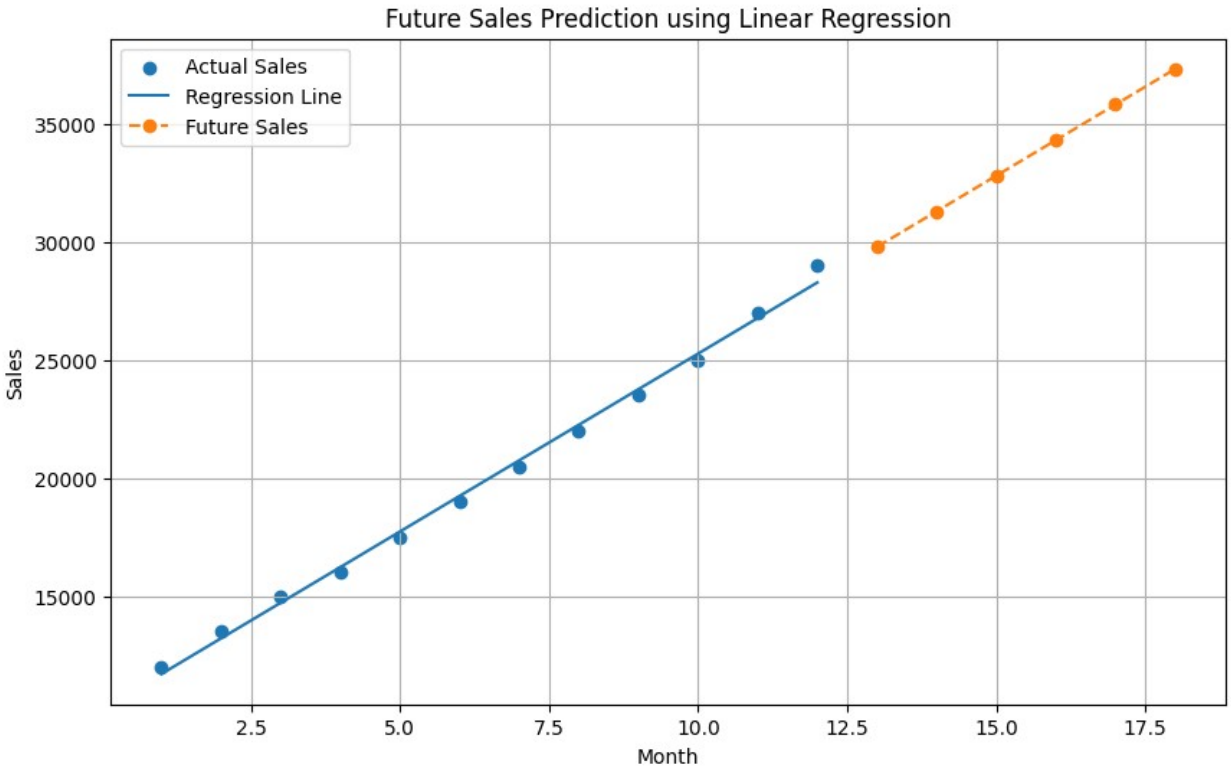
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Future Sales Prediction using Linear Regression")
```

```
plt.legend()
plt.grid(True)

plt.show()
```

```
Future Sales Prediction:
Month 13: 29795.45
Month 14: 31302.45
Month 15: 32809.44
Month 16: 34316.43
Month 17: 35823.43
Month 18: 37330.42
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(
```




```
import pandas as pd
import matplotlib.pyplot as plt

# Training dataset
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
```

```
    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

print("Training Data:")
print(df)

# Separate attributes and target
attributes = df.iloc[:, :-1].values
target = df.iloc[:, -1].values

# Initial most specific hypothesis
hypothesis = ['0'] * len(attributes[0])

# FIND-S algorithm
for i in range(len(attributes)):

    # Consider only positive examples
    if target[i] == 'Yes':

        for j in range(len(hypothesis)):

            if hypothesis[j] == '0':
                hypothesis[j] = attributes[i][j]

            elif hypothesis[j] != attributes[i][j]:
                hypothesis[j] = '?'

# Display final hypothesis
print("\nFinal Most Specific Hypothesis:")
print(hypothesis)

# ---------------------------------------
# GRAPH
# ---------------------------------------

# Convert hypothesis into graph values
# Specific value = 1
# Generalized '?' = 0

values = []

for h in hypothesis:
    if h == '?':
        values.append(0)
    else:
        values.append(1)
```

```
attribute_names = list(df.columns[:-1])

plt.figure(figsize=(9, 5))

plt.bar(attribute_names, values)

plt.xlabel("Attributes")
plt.ylabel("Specificity")
plt.title("FIND-S Algorithm - Final Hypothesis")

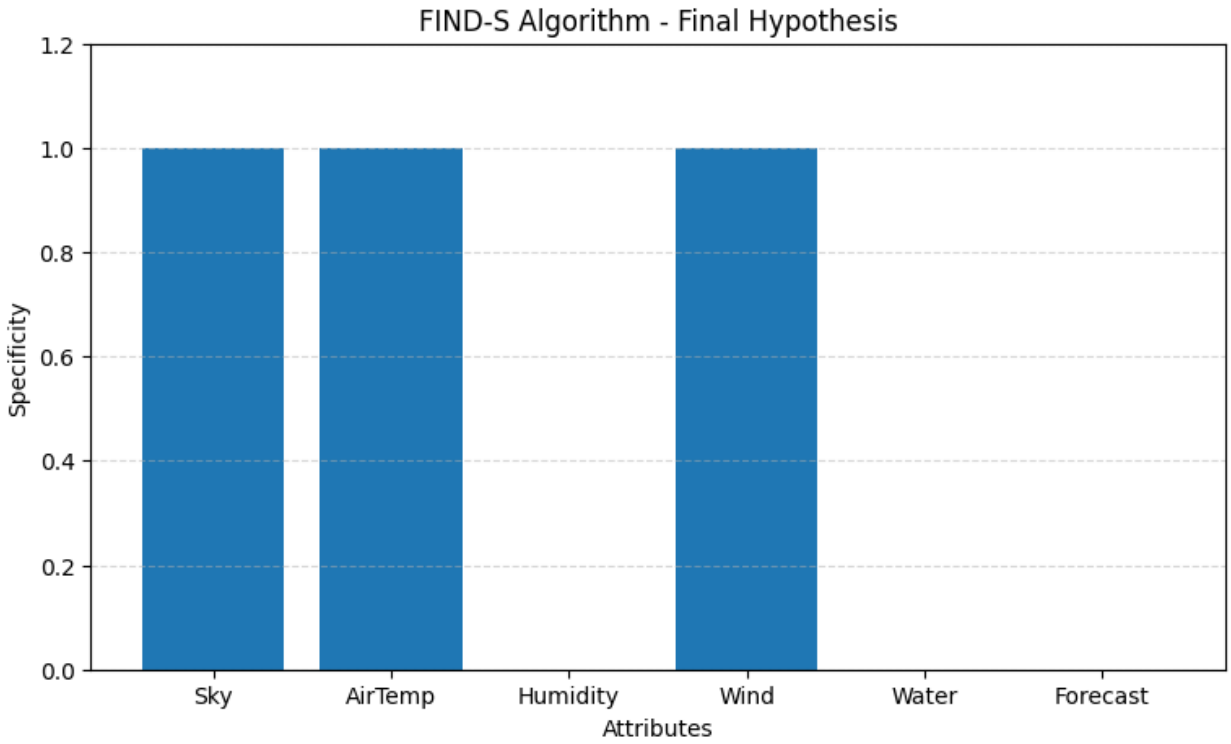
plt.ylim(0, 1.2)

plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```

```
Training Data:
     Sky AirTemp Humidity    Wind Water Forecast EnjoySport
0  Sunny    Warm   Normal  Strong  Warm     Same        Yes
1  Sunny    Warm     High  Strong  Warm     Same        Yes
2  Rainy    Cold     High  Strong  Warm   Change         No
3  Sunny    Warm     High  Strong  Cool   Change        Yes

Final Most Specific Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```



```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import OrdinalEncoder
from sklearn.metrics import accuracy_score

# Play Tennis dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
                'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
                'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
                    'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild',
                    'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                 'Normal', 'High', 'Normal', 'Normal', 'Normal',
                 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
             'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
             'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
                   'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

X = df.drop('PlayTennis', axis=1)
y = df['PlayTennis']

# Encode categorical attributes
encoder = OrdinalEncoder()
X_encoded = encoder.fit_transform(X)

# ID3 uses entropy and information gain.
# max_features=None allows all attributes to be considered.
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=0
)
model.fit(X_encoded, y)

# Display information gain for each root-level attribute
print("Information Gain at Root:")
root = model.tree_.feature[0]
root_name = X.columns[root]
root_entropy = -(y.value_counts(normalize=True) *
                 (y.value_counts(normalize=True).apply(
                     lambda p: __import__('math').log2(p)))).sum()
```

```
weighted_entropy = 0
for value in X[root_name].unique():
    subset = y[X[root_name] == value]
    probs = subset.value_counts(normalize=True)
    entropy = -(probs * probs.apply(
        lambda p: __import__('math').log2(p))).sum()
    weighted_entropy += len(subset) / len(y) * entropy

print(f"{root_name}: {root_entropy - weighted_entropy:.3f}")

# Predict a new sample
new_sample = pd.DataFrame([{
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'Normal',
    'Wind': 'Weak'
}])

new_encoded = encoder.transform(new_sample)
prediction = model.predict(new_encoded)

print("\nNew Sample:")
print(new_sample)
print("Predicted PlayTennis:", prediction[0])

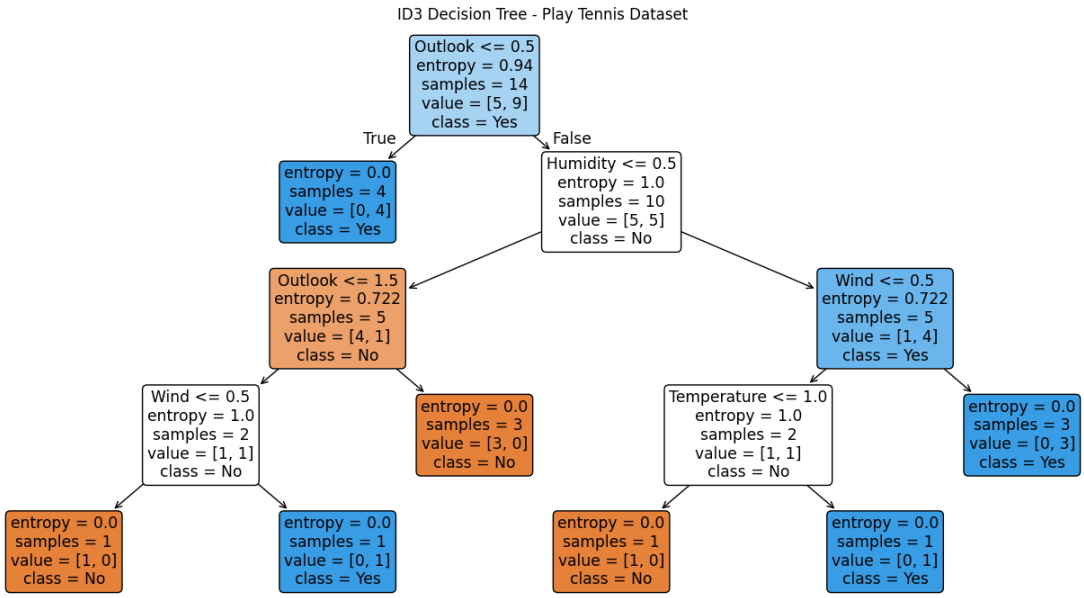
# Training accuracy
print("Training Accuracy:",
      round(accuracy_score(y, model.predict(X_encoded)) * 100, 2),
"%")

# Plot decision tree
plt.figure(figsize=(14, 7))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True
)
plt.title("ID3 Decision Tree - Play Tennis Dataset")
plt.tight_layout()
plt.show()
```

```
Information Gain at Root:
Outlook: 0.247
```

```
New Sample:
  Outlook Temperature Humidity  Wind
0   Sunny        Cool   Normal  Weak
```

```
Predicted PlayTennis: Yes
Training Accuracy: 100.0 %
```




```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale input features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build ANN and train using backpropagation
ann = MLPClassifier(
```

```
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    learning_rate_init=0.01,
    max_iter=1000,
    random_state=42
)

ann.fit(X_train, y_train)

# Predict test samples
y_pred = ann.predict(X_test)

# Print results
accuracy = accuracy_score(y_test, y_pred)
print("Actual Labels:   ", y_test)
print("Predicted Labels:", y_pred)
print("Test Accuracy:", round(accuracy * 100, 2), "%")

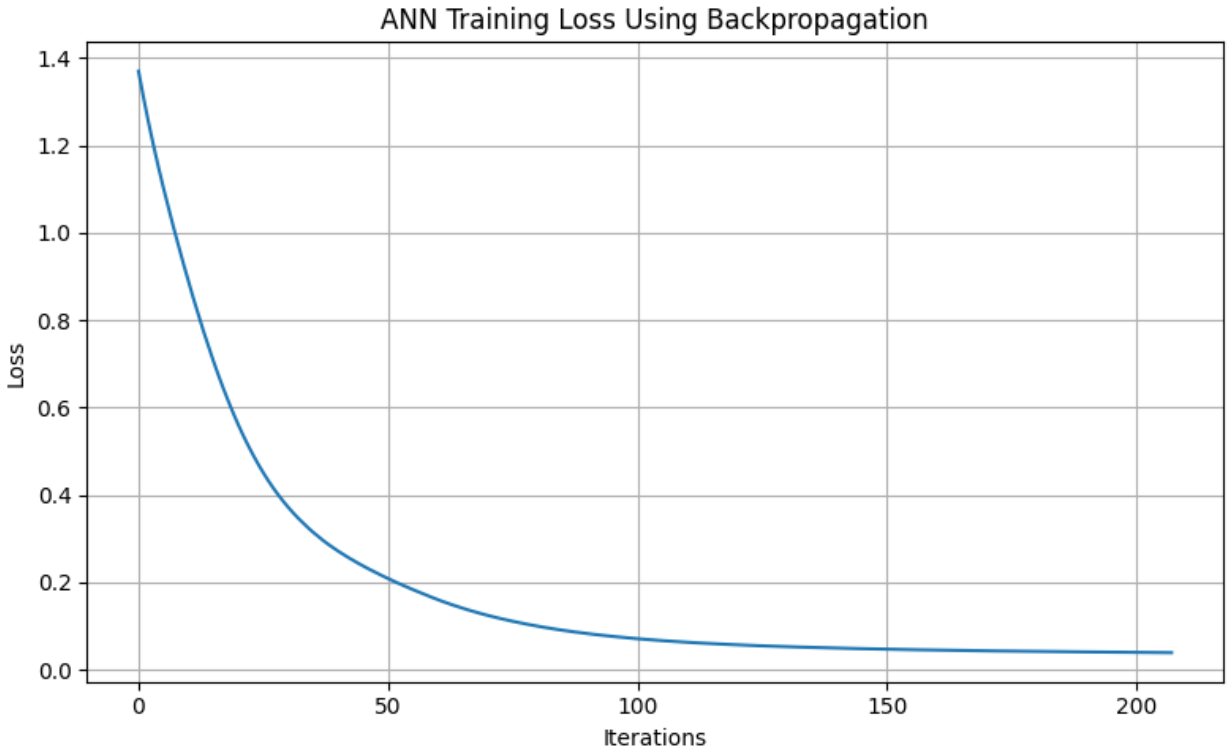
# Graph 1: Training loss
plt.figure(figsize=(8, 5))
plt.plot(ann.loss_curve_)
plt.title("ANN Training Loss Using Backpropagation")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)
plt.tight_layout()
plt.show()

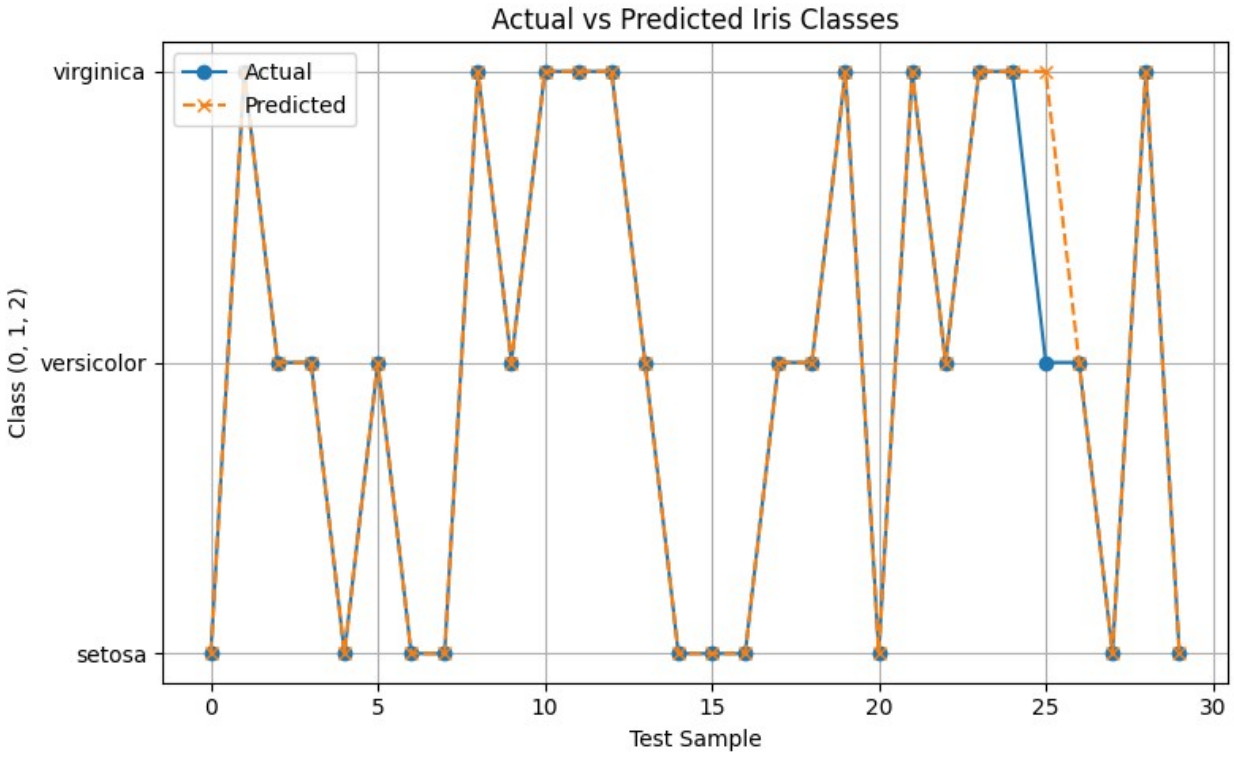
# Graph 2: Actual vs Predicted classes
plt.figure(figsize=(8, 5))
plt.plot(range(len(y_test)), y_test, 'o-', label='Actual')
plt.plot(range(len(y_pred)), y_pred, 'x--', label='Predicted')
plt.title("Actual vs Predicted Iris Classes")
plt.xlabel("Test Sample")
plt.ylabel("Class (0, 1, 2)")
plt.yticks([0, 1, 2], iris.target_names)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

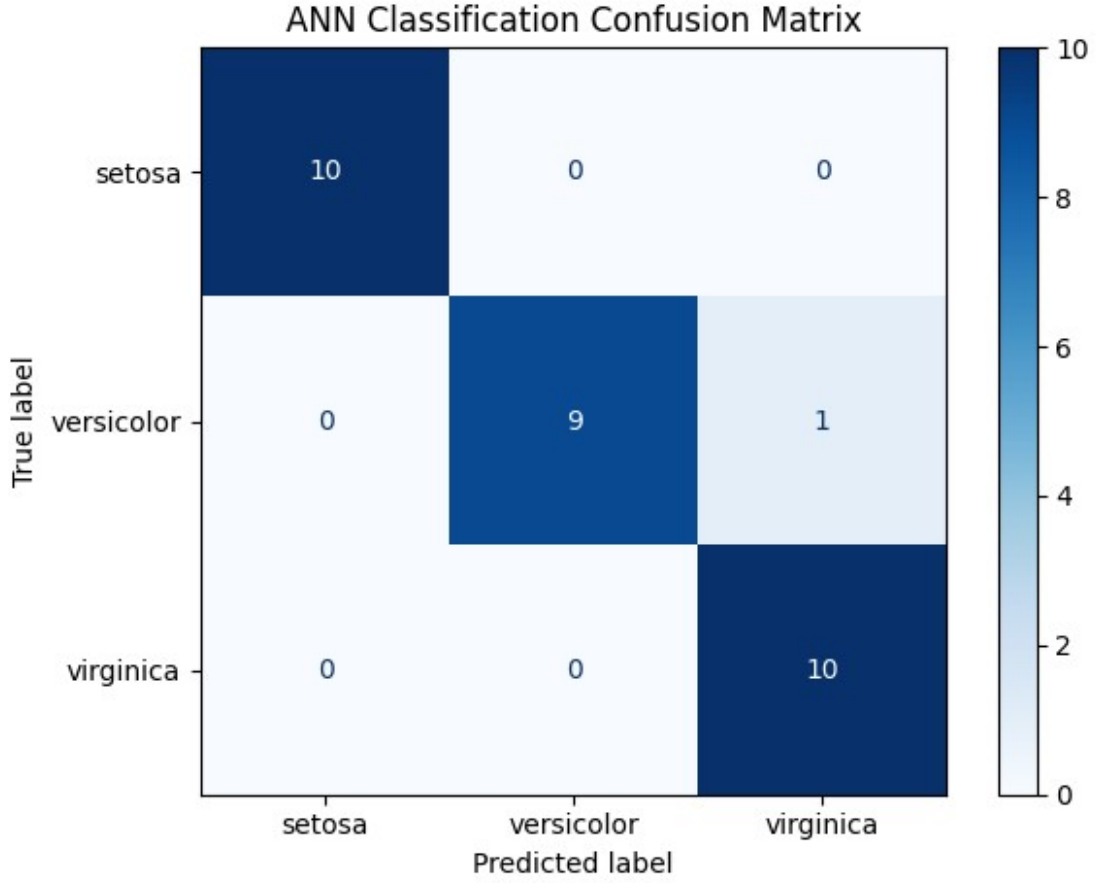
# Graph 3: Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)
disp.plot(cmap='Blues')
plt.title("ANN Classification Confusion Matrix")
```

```
plt.tight_layout()
plt.show()
```

```
Actual Labels:    [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1
1 0 2 0]
Predicted Labels: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2
1 0 2 0]
Test Accuracy: 96.67 %
```




Actual vs Predicted Iris Classes
Actual
Predicted
virginica
versicolor
setosa
Class (0, 1, 2)
0
5
10
15
20
25
30
Test Sample


ANN Classification Confusion Matrix
setosa
versicolor
virginica
True label
10
0
0
0
9
1
0
0
10
setosa
versicolor
virginica
Predicted label
10
8
6
4
2
0